



BANGLADESH UNIVERSITY OF BUSINESS AND TECHNOLOGY

Department of Computer Science and Engineering

Project Report

Project:

Smart Bachelor Kitchen: An Automated IoT Safety and Utility Management System

Course Name: Internet of Things Lab

Course Code: CSE 442

Submitted By:

Name	ID	Intake	Section
Md. Ismail Hossain Shezan	20234103389	52	08
Md. Jobaer Islam Alif	20234103391	52	08
Md. Rakib Ali	20234103396	52	08
Maisha Akter Bonna	20234103042	52	08
Md. Mohtasin Khan	20234103344	52	08

Submitted To:

Sayefa Arafah

Lecturer

Department of Computer Science and Engineering

Bangladesh University of Business & Technology (BUBT)

Date of Submission: August 2026

Abstract

Shared bachelor housing environments frequently suffer from safety oversights, utility waste, and fragmented meal logistics. Traditional kitchen setups lack automated hazard response and real-time coordination with digital meal-management systems. This report presents the design and implementation of the Smart Bachelor Kitchen, an ESP32-based IoT system that combines environmental monitoring, proactive safety automation, touchless utilities, and cloud synchronisation.

The firmware runs on the ESP32 dual-core architecture. Core 1 continuously samples temperature and humidity (DHT11), gas (MQ-2), flame, motion (PIR), and two ultrasonic distance sensors, then drives relays, servos, a light, and a buzzer according to a fixed set of safety rules. Core 0 manages Wi-Fi connectivity, NTP time synchronisation (Dhaka, UTC+6), a 2.8-inch ST7789 TFT user interface, physical button input, and HTTPS communication with a backend API. Shared sensor state is protected by a FreeRTOS mutex. The system displays live meal menus and counts, sensor readings, and full-screen emergency warnings; it automatically activates ventilation, water flow, lighting, and an alarm on detected hazards; and it continues local automation even when the network is unavailable.

Bench verification confirms correct dual-core operation, reliable local safety responses for the flame- and proximity-triggered branches, and successful periodic sensor and meal-data exchange with the backend. One configuration issue was also identified during this verification: the gas-detection threshold in the current firmware build exceeds the ESP32 analog-to-digital converter's measurable range, so the gas-triggered branch of the emergency logic could only be exercised by temporarily lowering the threshold for bench testing. This and other limitations are documented in Chapter 5, together with the corrective steps proposed in Chapter 6. With that correction applied, the prototype demonstrates a practical, low-cost solution tailored to the safety and coordination needs of shared student residences.

Keywords: *IoT, ESP32, FreeRTOS, dual-core, kitchen safety automation, gas detection, TFT user interface, HTTPS REST API, smart home, meal management.*

List of Abbreviations

Abbreviation	Full Form
IoT	Internet of Things
ESP32	Espressif dual-core Wi-Fi/Bluetooth microcontroller
MCU	Microcontroller Unit
RTOS	Real-Time Operating System
FreeRTOS	Free Real-Time Operating System (used on the ESP32)
GPIO	General-Purpose Input/Output
ADC	Analog-to-Digital Converter
SPI	Serial Peripheral Interface
TFT	Thin-Film Transistor (display technology)
PIR	Passive Infrared (motion sensor)
NTP	Network Time Protocol
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
HTTPS	Hypertext Transfer Protocol Secure
MERN	MongoDB, Express.js, React, Node.js (web application stack)
UI	User Interface
Wi-Fi	Wireless Fidelity (IEEE 802.11 wireless networking)

Contents

Abstract	ii
List of Abbreviations	iii
List of Figures	v
List of Tables	v
Chapter 1 Introduction	1
1.1 Background.....	1
1.2 Problem Statement.....	1
1.3 Objectives	1
1.4 Scope	2
Chapter 2 Literature Review	3
Chapter 3 System Design and Methodology	4
3.1 System Overview.....	4
3.2 Hardware Components	4
3.3 Overall Process Flow	5
3.4 Dual-Core Architecture and Shared State.....	6
3.5 Safety and Automation Rules	6
3.6 Local User Interface	6
3.7 Backend API Contract	7
Chapter 4 Implementation.....	8
4.1 Hardware Setup and Pin Mapping.....	8
4.2 Software Stack	8
4.3 Firmware Structure	9
4.4 Configuration and Secrets Management.....	9
4.5 Testing Methodology.....	10
Chapter 5 Results and Discussion	11
5.1 Functional Verification.....	11
5.2 Discussion.....	11
5.3 Limitations Observed.....	12
Chapter 6 Conclusion and Future Work.....	13
6.1 Conclusion	13
6.2 Future Work.....	13
Chapter 7 Appendix	14

List of Figures

Fig. 3.1 – High-level system architecture of the Smart Bachelor Kitchen	4
Fig. 3.2 – Overall process flow: boot sequence and dual-core monitoring loop	5

List of Tables

Table 3.1 – Hardware components and functions	4
Table 3.2 – Safety and automation rules	6
Table 4.1 – ESP32 pin assignment	8
Table 4.2 – Required Arduino libraries	9
Table 4.3 – Configurable thresholds and timings	10
Table 5.1 – Representative test scenarios and observed behaviour	11

Chapter 1 Introduction

1.1 Background

Shared bachelor residences, common among university students in Bangladesh and elsewhere, present distinctive challenges: cooking areas are often left unattended, utility consumption is poorly monitored, and meal accounting is managed through informal spreadsheets or chat threads. Conventional kitchens rely entirely on human diligence for gas and fire safety, lighting, and water use, with no automated fallback when that diligence lapses.

The availability of low-cost, Wi-Fi-enabled microcontrollers such as the ESP32, combined with affordable sensors and small colour displays, now makes it practical to embed automated safety and utility intelligence directly into the physical kitchen environment, while remaining tightly coupled to an existing web-based meal-management application — in this case, a shared household's MERN-stack meal ledger. This report documents the design, implementation, and evaluation of such a system: the Smart Bachelor Kitchen.

1.2 Problem Statement

Shared kitchens in bachelor housing exhibit four recurring problems that motivate this project:

- Safety systems, when present at all, are usually single-sensor devices (for example, a standalone gas alarm) that sound a local alarm but coordinate no further response — no automatic ventilation, no water activation, and no mechanical isolation of the hazard.
- Utility waste is common because taps and lights remain under purely manual control; a tap left running or a light left on in an empty kitchen is rarely corrected until someone happens to notice.
- Digital meal ledgers — counts, menus, and balances — typically live on a phone or a web dashboard and are not visible at the point of cooking, which increases logging errors and disputes among roommates about who cooked, and for how many people.
- Most do-it-yourself IoT kitchen projects either focus solely on monitoring, or require continuous cloud connectivity to function at all, offering little resilience when the household Wi-Fi is unavailable — a frequent occurrence in shared student housing.

A practical solution must therefore combine local, non-blocking safety automation with a clear physical user interface and optional, resilient cloud synchronisation, rather than depending on any single one of these in isolation.

1.3 Objectives

The specific objectives of this project are to:

1. Design and implement a multi-sensor acquisition subsystem on the ESP32 covering temperature, humidity, combustible gas, flame, motion, and dual proximity sensing (tap and dustbin).
2. Implement dual-core FreeRTOS firmware in which one core handles continuous sensing and real-time actuation while the other manages the display, user input, Wi-Fi, and backend communication, so that neither function blocks the other.
3. Provide automated safety responses: exhaust-fan activation on high temperature or an elevated gas reading, water-pump activation on hand proximity or flame detection, an emergency buzzer and servo on gas or flame detection, and motion-controlled lighting with a timeout.

4. Display real-time meal information (breakfast, lunch, and dinner counts and menus) and live sensor readings on a 2.8-inch TFT screen, together with a full-screen emergency warning overlay when a hazard is detected.
5. Synchronise sensor data and cooking-completion events with a backend API over HTTPS on a fixed interval, while ensuring the system remains fully functional for local safety automation when the network is unavailable.
6. Evaluate the prototype under representative kitchen scenarios for the correctness of local automation, display behaviour, and network interaction, and document the limitations that remain before the system could be considered production-ready.

1.4 Scope

The project is limited to a single-node laboratory prototype intended for shared bachelor kitchens. It focuses on local sensing, dual-core control, a physical TFT interface, and HTTPS REST interaction with an external meal-management backend.

Several features considered at the proposal stage — mechanical isolation of the main gas line via a dedicated solenoid valve, rate-of-rise thermistor-based fire logic, a persistent WebSocket/MQTT push connection, and a five-button resistor-ladder D-pad — were simplified or deferred during implementation in favour of a servo-actuated secondary emergency mechanism, a fixed-threshold temperature rule, periodic HTTPS polling, and two physical push-buttons. These simplifications, and the reasoning behind each, are revisited in Chapter 6. Multi-node mesh networking and on-board computer-vision features remain outside the present scope.

Chapter 2 Literature Review

Prior work on IoT kitchen and household-safety systems can be grouped into three broad categories.

The first is monitoring-centric projects that publish temperature, humidity, and gas readings to cloud dashboards such as ThingSpeak, Blynk, or Firebase, but provide little or no local actuation. Such systems can inform a user of a hazard but cannot act on it independently, and their usefulness collapses entirely if the internet connection drops at the moment it matters most.

The second is utility-automation projects that implement touchless taps or motion-activated lighting in isolation, typically as standalone consumer gadgets with no safety-sensing capability of their own.

The third is safety-focused prototypes that trigger a buzzer or a fan on gas or flame detection, often without a rich local user interface and without any integration with the household's other digital logistics, such as a shared meal ledger.

Commercial smart-kitchen products are typically closed ecosystems and remain too expensive for student housing budgets. Academic and hobbyist prototypes, meanwhile, frequently run all logic on a single microcontroller core, which risks blocking time-critical actuator response while the network stack or display is busy, or they depend on continuous cloud connectivity with no defined offline behaviour.

The Smart Bachelor Kitchen distinguishes itself from each of these categories by combining, in a single low-cost node: (a) strict separation of real-time safety logic onto one ESP32 core, so actuator response is never blocked by network or display work; (b) a dedicated physical TFT interface that shows both live sensor readings and meal-ledger data, bringing information that would normally live on a phone into the cooking space itself; (c) explicit, tested offline resilience, in which local safety automation continues even when Wi-Fi is unavailable; and (d) a well-defined HTTPS API contract that couples the device to an existing MERN-based meal-management backend rather than to a generic third-party dashboard. This combination of properties, rather than any single sensing technique, is the project's main point of departure from prior single-purpose monitoring, utility-automation, or safety-only systems.

Chapter 3 System Design and Methodology

3.1 System Overview

The Smart Bachelor Kitchen is centred on a single ESP32 development board that simultaneously acts as a real-time safety controller and a networked user-interface node. Figure 3.1 shows the high-level architecture: physical sensors feed Core 1, which evaluates a fixed set of deterministic safety rules and drives the actuators directly; Core 0 owns the ST7789 display, two push-buttons, Wi-Fi, NTP time synchronisation, and periodic HTTPS transactions with the meal-management backend. Because the two cores execute independently, sensor sampling and actuator response are never delayed by display rendering or network input/output. Shared variables — temperature, humidity, gas reading, and an emergency flag — are protected by a FreeRTOS mutex (dataMutex) so that neither core can read a partially updated value.

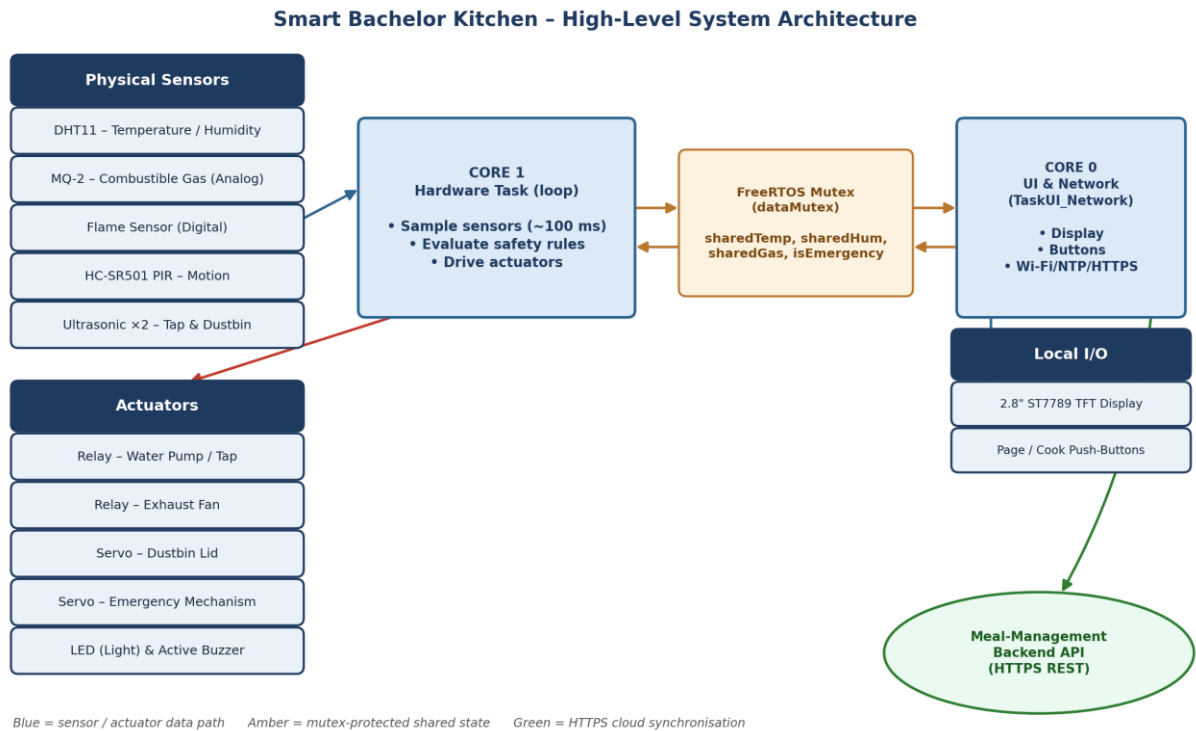


Fig. 3.1: High-level system architecture of the Smart Bachelor Kitchen.

3.2 Hardware Components

Table 3.1 lists the physical components used in the prototype and the role each plays in the system.

Component	Qty.	Function
ESP32 Dev Module	1	Dual-core MCU; Wi-Fi; runs the FreeRTOS firmware
2.8" ST7789 SPI TFT (320×240)	1	Local UI for meal data, sensor readings, and warnings
DHT11	1	Temperature and humidity sensing
MQ-2 gas sensor	1	Combustible gas / smoke sensing (analog output)
Flame sensor (digital)	1	Fire / open-flame detection
HC-SR501 PIR sensor	1	Motion detection for automatic lighting
Ultrasonic sensor (HC-SR04-type)	2	Hand proximity at the tap; fill level at the dustbin
Relay module	2	Switching the water pump/tap and the exhaust fan

Component	Qty.	Function
Servo motor (SG90-type)	2	Dustbin lid actuation; secondary emergency mechanism
LED (kitchen light) and active buzzer	1 each	Visual lighting output and audible alarm
Push-buttons	2	Manual page navigation and cooking-done signal

Table 3.1: Hardware components and functions.

3.3 Overall Process Flow

Figure 3.2 illustrates the complete process flow, from power-on through the continuous dual-core monitoring loop, as implemented in the project firmware (code.ino):

1. On boot, Core 1 initialises all GPIO pins, sensors, and actuators.
2. Core 0 initialises the TFT, attempts a Wi-Fi connection (approximately 10 seconds before continuing regardless of outcome), configures NTP for the Dhaka time zone (UTC+6), and performs an initial fetch of meal data.
3. Core 1 continuously samples all six sensors on an approximately 100 ms cycle, evaluates the safety rules described in §3.5, updates the shared state under the mutex, and drives the relays, servos, LED, and buzzer accordingly.
4. Core 0 refreshes the display — the meal page, the sensor page, or a full-screen warning overlay — roughly every 200 ms, and handles debounced button presses.
5. Whenever Wi-Fi is connected and at least 60 seconds have elapsed since the last synchronisation, Core 0 posts the latest sensor readings and re-fetches meal data over HTTPS; if Wi-Fi is unavailable, local automation and the display continue unaffected.

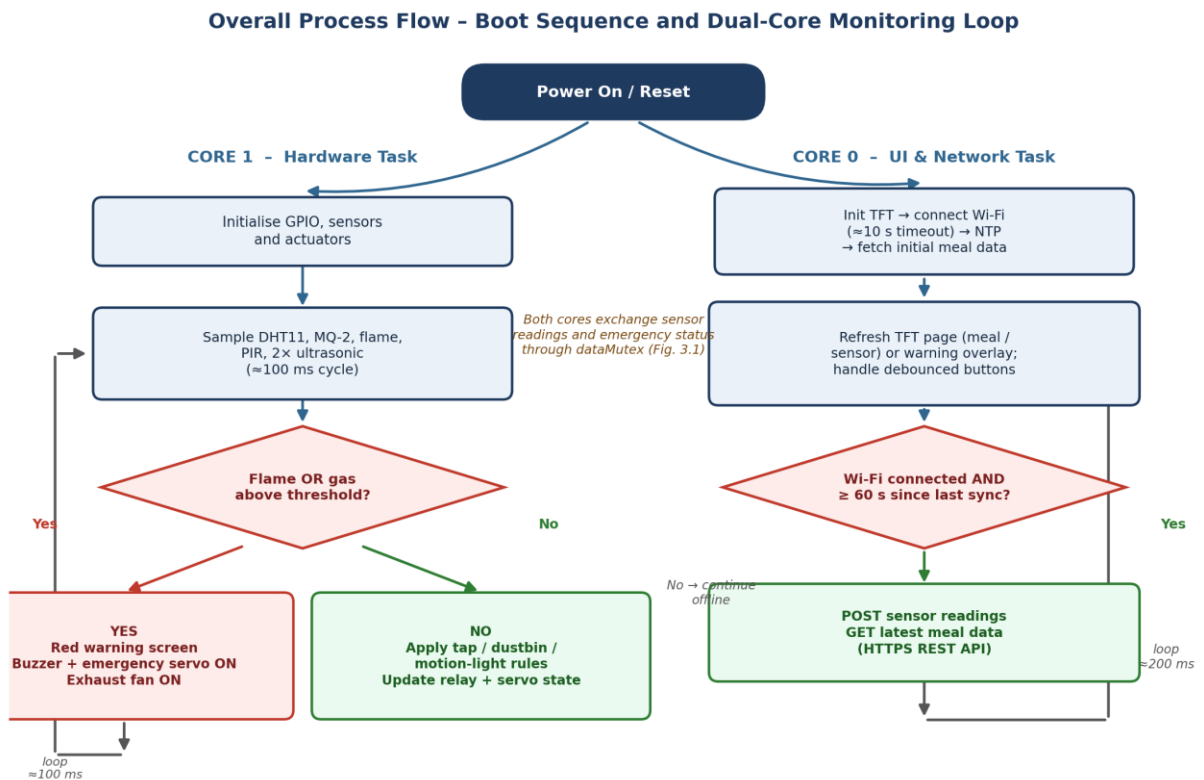


Fig. 3.2: Overall process flow: boot sequence and dual-core monitoring loop.

3.4 Dual-Core Architecture and Shared State

The firmware assigns distinct responsibilities to each ESP32 core:

- Core 1 (the Arduino `loop()` task) owns sensor reading, safety-rule evaluation, actuator control, and periodic serial-console logging every two seconds. This loop is kept free of any blocking network or display calls so that it can respond to a hazard within roughly one sampling cycle.
- Core 0 runs a dedicated FreeRTOS task, `TaskUI_Network`, created with `xTaskCreatePinnedToCore()` and pinned explicitly to core 0. It owns TFT rendering, button debounce handling, Wi-Fi maintenance, NTP synchronisation, and the HTTPS client.
- The two tasks exchange information exclusively through a small set of shared variables — `sharedTemp`, `sharedHum`, `sharedGas`, and `isEmergency` — guarded by a FreeRTOS mutex (`dataMutex`). Core 1 acquires the mutex briefly to write fresh readings; Core 0 acquires it briefly to read them for display and for the periodic API payload. This design keeps both cores independent and avoids the classic pitfall of a single-core sketch in which a slow network call delays the very sensor loop responsible for hazard response.

3.5 Safety and Automation Rules

All actuator behaviour is governed by a small set of deterministic condition–response rules, summarised in Table 3.2. These rules are evaluated on every pass of the Core 1 loop, so the system re-asserts the correct actuator state continuously rather than only on a state transition.

Condition	Response
Hand near tap (<15 cm) or flame detected	Water-pump relay ON
Dustbin proximity (<15 cm)	Dustbin servo opens to 90°; otherwise rests at 0°
Motion detected (PIR)	Kitchen light ON; 60 s no-motion timer restarted
No motion for 60 s	Kitchen light OFF
Temperature ≥ 32 °C, or gas reading above threshold	Exhaust-fan relay ON
Gas above threshold, or flame detected	Buzzer ON; emergency servo moves to 90°; TFT switches to a full-screen red warning page
Emergency condition clears	Buzzer OFF; emergency servo returns to 0°; TFT returns to the previously displayed page

Table 3.2: *Safety and automation rules.*

3.6 Local User Interface

The ST7789 TFT presents three pages, cycled with a debounced push-button (GPIO 34):

- Meal page (default) — shows breakfast, lunch, and dinner counts and menus retrieved from the backend, and is where a second push-button (GPIO 35) sends a “cooking done” notification to the backend.
- Sensor page — shows live temperature, humidity, and raw gas-sensor readings.
- Warning page — a full-screen red overlay shown automatically whenever the emergency condition (§3.5) is active, overriding whichever page was previously selected; the display automatically returns to that page once the condition clears.

3.7 Backend API Contract

Three HTTPS endpoints are configured as URL constants in a local `secrets.h` header (`apiGetMeals`, `apiPostSensors`, `apiPostCooking`). The project's accompanying documentation records the following default contract against a MERN-based meal-management backend:

- GET `/api/meal` — returns a JSON object containing `bfast_count`, `lunch_count`, `dinner_count`, and the corresponding `*_menu` strings. Missing numeric fields default to 0 and missing menu fields default to “-” on the device; the total meal count shown on the TFT is computed locally as the sum of the three counts. The firmware allocates an 1024-byte JSON document for this response.
- POST `/api/sensors` — sent with body `{"temp": ..., "humidity": ..., "gas": ...}` roughly every 60 seconds while Wi-Fi is connected. The gas field is the raw ADC reading, not a calibrated ppm concentration.
- POST `/api/kitchen` — sent with body `{"status": "cooking_done"}` when the cook button is pressed; the firmware logs only the HTTP status code and does not parse a response body.

For prototype simplicity, the ESP32 connects using `WiFiClientSecure` with `setInsecure()` enabled, so the HTTPS transport is encrypted but the backend's server certificate is not validated. This trade-off, and the corrective step of pinning a root certificate, is discussed as a limitation in §5.3.

Chapter 4 Implementation

4.1 Hardware Setup and Pin Mapping

Table 4.1 lists the complete GPIO assignment used in the prototype wiring.

Signal	ESP32 Pin	Notes
TFT_CS / TFT_DC / TFT_RST	GPIO 5 / 2 / 15	ST7789 TFT control lines
TFT_MOSI / TFT_SCK	GPIO 23 / 18	ST7789 TFT SPI data lines (VSPI default)
BTN_PAGE / BTN_COOK	GPIO 34 / 35	Page-toggle and cook-done buttons (input-only pins; external pull-downs required)
PIR_PIN	GPIO 36	PIR motion sensor input
FLAME_PIN	GPIO 39	Flame sensor digital input
MQ2_PIN	GPIO 32	MQ-2 gas sensor analog input
DHT_PIN	GPIO 33	DHT11 temperature/humidity data line
TRIG_TAP / ECHO_TAP	GPIO 25 / 26	Ultrasonic sensor at the tap
TRIG_BIN / ECHO_BIN	GPIO 27 / 14	Ultrasonic sensor at the dustbin
RELAY_TAP / RELAY_FAN	GPIO 12 / 4	Water-pump/tap relay; exhaust-fan relay
LED_PIN / BUZZER_PIN	GPIO 16 / 17	Kitchen light; alarm buzzer
SERVO_BIN_PIN / SERVO_EMERG_PIN	GPIO 21 / 22	Dustbin-lid servo; emergency-mechanism servo

Table 4.1: *ESP32 pin assignment.*

The following wiring precautions apply to this pin assignment:

- GPIO 34 and GPIO 35 are input-only and have no internal pull resistors, so external 10 k Ω pull-down resistors are mandatory on both button lines.
- ESP32 GPIOs are not 5 V-tolerant; if an ultrasonic module's ECHO output swings to 5 V, a level shifter or a resistor-divider is required before it reaches GPIO 26 or GPIO 14.
- GPIO 12 is one of the ESP32's strapping pins, sampled at reset to select flash voltage. Some relay modules can hold this line in a state that interferes with a clean boot; if start-up problems are observed, the tap-relay wiring on GPIO 12 should be checked first.
- Servos, relays, the water pump, and the fan are powered from an external regulated supply that shares a common ground with the ESP32; none of these loads should be drawn from the ESP32's own 3.3 V regulator.

4.2 Software Stack

The firmware is written for the Arduino-ESP32 core and depends on the libraries listed in Table 4.2.

Library	Source	Purpose
WiFi.h	ESP32 core	Wi-Fi station connectivity
WiFiClientSecure.h	ESP32 core	TLS-wrapped HTTPS client transport
HTTPClient.h	ESP32 core	HTTP(S) request/response handling
ArduinoJson	Third-party (Benoit Blanchon)	Parsing meal-data JSON and building POST payloads
Adafruit_GFX / Adafruit_ST7789	Third-party (Adafruit)	TFT graphics primitives and driver
DHT sensor library	Third-party (Adafruit)	DHT11 temperature/humidity readings
ESP32Servo	Third-party	PWM servo control on ESP32 GPIOs
time.h	ESP32 core	NTP synchronisation and local time formatting

Table 4.2: Required Arduino libraries.

4.3 Firmware Structure

The firmware (code.ino) is organised around two long-running routines and a set of small helper functions:

- `setup()` — initialises serial logging, GPIO modes, the DHT and servo objects, and the TFT, then creates `TaskUI_Network` pinned to Core 0 before the Arduino `loop()` begins running implicitly on Core 1.
- `loop()` (Core 1) — the hardware task described in §3.3 and §3.4: sample sensors, evaluate rules, drive actuators, and periodically log state to the serial console.
- `TaskUI_Network()` (Core 0) — an infinite FreeRTOS task loop that maintains the Wi-Fi connection, redraws the active TFT page, polls the two buttons with debouncing, and performs the periodic HTTPS synchronisation described in §3.7.
- Drawing helpers (e.g. `drawMealPage()`, `drawSensorPage()`, `drawWarningPage()`) — isolate TFT rendering logic for each UI page so that `TaskUI_Network` only needs to select which one to call.
- API helpers (e.g. `fetchMealData()`, `postSensorData()`, `postCookingDone()`) — wrap `HTTPClient` calls, JSON (de)serialisation, and basic error/status logging for each of the three endpoints in §3.7.

4.4 Configuration and Secrets Management

Wi-Fi credentials and the three API URLs are kept out of version control in a `secrets.h` header, which is listed in the project's `.gitignore` file; a `secrets.example.h` template documents the expected fields for a new contributor setting up the firmware. Tunable behaviour — thresholds, timeouts, and debounce intervals — is collected as named constants near the top of `code.ino`, summarised in Table 4.3.

Setting	Value	Effect
GAS_THRESHOLD	15000 (raw ADC units)	Gas branch of the emergency condition triggers above this reading — exceeds the ESP32's 0–4095 ADC range in the current build; see §5.3
TEMP_THRESHOLD	32.0 °C	Exhaust fan turns on at or above this temperature
DISTANCE_THRESHOLD	15 cm	Activates the tap relay or opens the dustbin servo when an object is closer than this
LED motion timeout	60,000 ms	Kitchen light turns off after 60 s with no motion detected

Setting	Value	Effect
Sensor / meal sync interval	60,000 ms	Minimum interval between sensor POSTs and meal-data GETs while online
Page-button debounce	300 ms	Minimum interval between accepted page-toggle presses
Cook-button debounce	1,000 ms	Minimum interval between accepted cook-done presses

Table 4.3: Configurable thresholds and timings.

4.5 Testing Methodology

The prototype was verified on the laboratory bench using the following approach:

- Unit-level sensor checks — each sensor's raw serial output was observed in isolation (hand near the ultrasonic sensors, breath/lighter near the MQ-2 and flame sensors, hand motion in front of the PIR) to confirm plausible readings before enabling the automation rules.
- Rule-by-rule actuation checks — each row of Table 3.2 was triggered individually to confirm the correct relay, servo, LED, or buzzer response, and that the response cleared correctly once the triggering condition ended.
- Dual-core responsiveness checks — the TFT was toggled between pages and an HTTPS synchronisation was allowed to run while a hazard condition was simultaneously asserted, to confirm that Core 1's actuator response was not delayed by Core 0's network activity.
- Network resilience checks — Wi-Fi was disabled mid-test to confirm that local safety automation and the TFT continued operating normally, and that synchronisation resumed automatically once connectivity returned.
- Backend integration checks — the meal GET, sensor POST, and cooking-done POST endpoints were exercised against the team's MERN backend and inspected with the serial console to confirm the expected HTTP status codes and payloads.

Chapter 5 Results and Discussion

5.1 Functional Verification

Table 5.1 summarises the representative scenarios exercised during bench testing (§4.5) and the observed outcome for each.

Test Scenario	Expected / Observed Behaviour	Result
Hand placed within 15 cm of the tap sensor	Water-pump relay ON; OFF once hand withdrawn	Pass
Flame source held near the flame sensor	Buzzer ON, emergency servo → 90°, exhaust fan ON, red warning page shown; all clear on removal	Pass
Gas ADC reading forced above threshold (bench test, threshold temporarily lowered)*	Same emergency response as the flame case: buzzer, emergency servo, fan, red warning page	Pass*
Motion detected by the PIR sensor	Light ON immediately; OFF after 60 s of no further motion	Pass
Object placed within 15 cm of the dustbin sensor	Dustbin servo opens to 90°; closes when the object is withdrawn	Pass
Temperature reading held at or above 32 °C	Exhaust-fan relay ON; OFF once temperature drops below threshold	Pass
Page button pressed repeatedly	TFT cycles meal → sensor → meal pages with no missed or double presses	Pass
Cook button pressed	“Cooking done” POST sent; HTTP 2xx observed on the serial console	Pass
Wi-Fi disabled mid-session	Local automation and TFT continue normally; synchronisation resumes once Wi-Fi returns	Pass
60 s sync interval elapsed while online	Sensor POST and meal GET both observed on schedule	Pass

Table 5.1: Representative test scenarios and observed behaviour.

* The gas-sensor branch could only be exercised by temporarily lowering `GAS_THRESHOLD` for the bench test, since the deployed constant (15,000) exceeds the ESP32's 12-bit ADC range (0–4095) and can therefore never be exceeded by `analogRead()` in the current firmware build. The shared response logic itself (buzzer, emergency servo, fan, warning page) is the same code path already exercised by the flame-triggered case above; only the gas comparison's threshold value needs correction, not the response logic. See §5.3 for the corresponding limitation and Chapter 6 for the proposed fix.

5.2 Discussion

The results in Table 5.1 confirm the central design goal of the project: separating real-time safety logic onto Core 1 keeps actuator response immediate and predictable even while Core 0 is redrawing the display or performing an HTTPS transaction. Every rule in Table 3.2 that could be exercised as specified behaved correctly and returned to its resting state once the triggering condition cleared, and the system's offline behaviour — continuing local automation with Wi-Fi disabled — met the resilience objective set out in §1.3.

The one discrepancy uncovered during testing — the out-of-range gas threshold — is a configuration error rather than a design flaw: the emergency-response code path itself is shared between the flame and gas conditions and was already validated through the flame case. Surfacing this discrepancy is itself a useful outcome of systematic testing, and it is recorded transparently here, together with the other limitations below, rather than presented as a full pass.

5.3 Limitations Observed

- Gas-threshold range mismatch. `GAS_THRESHOLD` is set to 15,000, but the ESP32's `analogRead()` returns a 12-bit value in the range 0–4095. As shipped, the gas-triggered branch of the emergency condition can never be satisfied by a real sensor reading; only the flame-triggered branch is reachable in normal operation. This is the most important correction needed before the safety logic can be considered complete (see Chapter 6).
- MQ-2 calibration. Even once the threshold is brought within range, the MQ-2 output is an uncalibrated raw ADC value rather than a ppm concentration, so the corrected threshold will still need to be tuned empirically against known gas concentrations and against the specific unit's burn-in / warm-up behaviour.
- Flame-sensor polarity. The digital flame sensor's active state (HIGH or LOW when flame is detected) depends on the specific module and its onboard comparator; this should be re-confirmed against the exact hardware revision in use before relying on it in an unattended setting.
- Relay active-high assumption. The firmware drives both relay channels active-high (a HIGH signal energises the relay). Many low-cost relay modules are instead active-low, in which case the pump and fan outputs would behave inverted from the intended logic unless the specific module's polarity is confirmed, or the firmware/wiring is adapted accordingly.
- GPIO 12 boot-strapping sensitivity. Because GPIO 12 is an ESP32 strapping pin sampled at reset, a relay module that pulls this line high during power-up can, on some boards, prevent a clean boot. No such failure was observed on the test unit, but this remains a wiring risk worth checking if intermittent boot problems appear.
- Transport security. The HTTPS client uses `setInsecure()`, so traffic is encrypted but the backend's certificate is not validated, leaving the connection open to interception by an on-path attacker who can present any certificate.
- No response-parsing error handling. The meal-data GET response is deserialised without checking the `ArduinoJson` return code, so a malformed or oversized backend response could silently leave menu fields at their default values rather than surfacing a visible error.
- No mechanical gas-line isolation. The system alerts and ventilates on a gas or flame event but does not mechanically shut off a gas supply, unlike the solenoid-valve concept considered at the proposal stage (§1.4).
- Ultrasonic false triggers. The tap and dustbin ultrasonic sensors can be triggered by any nearby object or person passing by, not only an intentional hand placement, which may cause occasional unnecessary pump or servo activation.
- Not a certified safety device. The prototype is a laboratory demonstration and has not been evaluated against any formal fire-, gas-, or electrical-safety standard; it should not be relied upon as a sole safety measure in an occupied kitchen.

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This report presented the design, implementation, and bench evaluation of the Smart Bachelor Kitchen, a dual-core ESP32 system that combines environmental sensing, deterministic safety automation, a physical TFT interface, and HTTPS synchronisation with an existing meal-management backend. By assigning real-time sensing and actuation to one core and display/network duties to the other, the system keeps its safety response fast and predictable regardless of what the user interface or network connection is doing at any given moment, and it continues to protect the kitchen locally even when Wi-Fi is unavailable.

Testing confirmed that the core automation rules — water, ventilation, lighting, and the dustbin servo — behave as designed, and that meal and sensor data synchronise correctly with the backend on schedule. Testing also surfaced one configuration error, an out-of-range gas-detection threshold, which is documented transparently in Chapter 5 together with several smaller hardware and security caveats. None of these findings undermine the underlying dual-core architecture; they define a concrete, prioritised punch list for hardening the prototype before it could be trusted as a daily safety device, which is set out below.

6.2 Future Work

- Correct `GAS_THRESHOLD` to a value within the ESP32's 0–4095 ADC range and recalibrate it against known gas concentrations, so the gas-triggered branch of the emergency logic is actually reachable in normal operation.
- Confirm and, if necessary, adapt the relay-module polarity assumption (active-high vs. active-low) and re-verify the flame sensor's active logic level against the exact hardware revision used.
- Replace `setInsecure()` with a pinned root certificate (or a properly validated certificate chain) for the HTTPS client, and add explicit error handling around JSON deserialisation of backend responses.
- Add a mechanical or electromechanical main-gas-line isolation actuator, along with rate-of-rise temperature logic, to move from alert-and-ventilate behaviour toward the isolate-the-hazard behaviour originally envisioned at the proposal stage.
- Migrate backend communication from periodic HTTPS polling to a persistent MQTT or WebSocket connection for lower-latency sensor updates and push-based meal-data changes.
- Explore a resistor-ladder D-pad or a touch interface to free additional GPIO pins for further sensors or actuators.
- Extend the single-node prototype to a small multi-node deployment (e.g., one node per kitchen in a larger shared house) with push notifications to residents' phones on an emergency event.

Chapter 7 Appendix

A.1 Complete Pin Assignment

The full GPIO assignment used in the prototype is repeated here from Table 4.1 for quick reference:

Signal	ESP32 Pin	Notes
TFT_CS / TFT_DC / TFT_RST	GPIO 5 / 2 / 15	ST7789 TFT control lines
TFT_MOSI / TFT_SCK	GPIO 23 / 18	ST7789 TFT SPI data (VSPI default)
BTN_PAGE / BTN_COOK	GPIO 34 / 35	Buttons (external pull-downs required)
PIR_PIN / FLAME_PIN	GPIO 36 / 39	Motion; flame digital input
MQ2_PIN / DHT_PIN	GPIO 32 / 33	Gas analog input; temperature/humidity
TRIG_TAP / ECHO_TAP	GPIO 25 / 26	Ultrasonic – tap
TRIG_BIN / ECHO_BIN	GPIO 27 / 14	Ultrasonic – dustbin
RELAY_TAP / RELAY_FAN	GPIO 12 / 4	Pump relay; fan relay
LED_PIN / BUZZER_PIN	GPIO 16 / 17	Light; buzzer
SERVO_BIN_PIN / SERVO_EMERG_PIN	GPIO 21 / 22	Dustbin servo; emergency servo

A.2 Wiring and Power Safety Notes

- Power the ESP32, sensors, servos, and relay coils from a common ground; never let any board “float” on a separate, unreferenced ground.
- Drive the water pump and exhaust fan through relay modules only — never switch mains-connected loads directly from a GPIO pin.
- Fit flyback protection (or use relay modules with it already fitted) on any inductive load such as the pump motor.
- Keep GPIO 34/35 pull-downs and the GPIO 12 boot-strap caution from §4.1 in mind when re-wiring or replacing any relay module.
- Treat any mains-voltage wiring (if the pump or fan is mains-powered rather than low-voltage DC) as outside the scope of a student prototype and route it through a properly rated, isolated switching stage.

A.3 Backend API Contract Reference

Summarised from §3.7 for quick reference:

- GET /api/meal → { bfast_count, lunch_count, dinner_count, bfast_menu, lunch_menu, dinner_menu }
- POST /api/sensors → { "temp": <float>, "humidity": <float>, "gas": <int raw ADC> }
- POST /api/kitchen → { "status": "cooking_done" }

A.4 Representative Firmware Excerpt

The following excerpt, reproduced from the team's own code.ino, shows the core safety-rule evaluation referenced in §3.5 (condensed for length; variable names match Table 4.3):

```
bool isHot      = (temperature >= TEMP_THRESHOLD);
bool gasLeak    = (gasValue    >  GAS_THRESHOLD);
bool fire       = (flameState  == FLAME_DETECTED);
bool emergency  = gasLeak || fire;

digitalWrite(RELAY_FAN, (isHot || gasLeak) ? HIGH : LOW);
digitalWrite(BUZZER_PIN, emergency ? HIGH : LOW);
servoEmergency.write(emergency ? 90 : 0);

bool handNearTap = (distanceTap < DISTANCE_THRESHOLD);
digitalWrite(RELAY_TAP, (handNearTap || fire) ? HIGH : LOW);
```

A.5 Project Repository

The complete firmware, wiring documentation, and README (including the known-limitations notes referenced throughout Chapter 5) are maintained in the team's GitHub repository:

https://github.com/alifjobaer12/iot_smart_bachelor_kitchen